

面向Agentic的OLAP 架构探索

演讲嘉宾：冯吕 微信技术架构部

目录

CONTENT

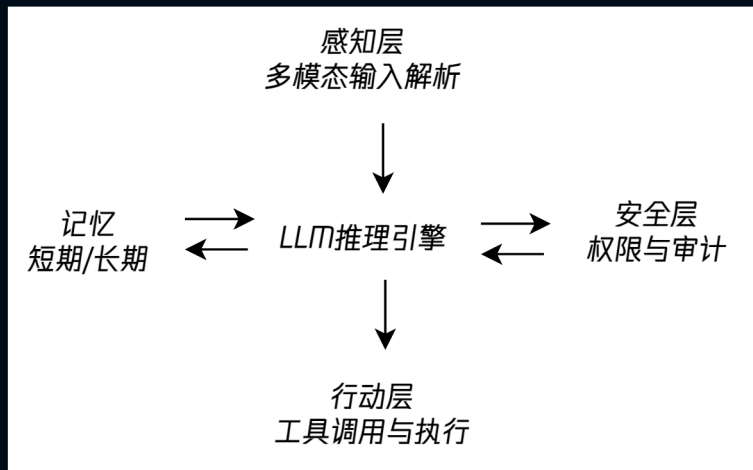
- 01 OLAP 遇见 Agentic
- 02 Agent 可观测性探索
- 03 Agent 记忆底座探索
- 04 Agent 访问保护

01

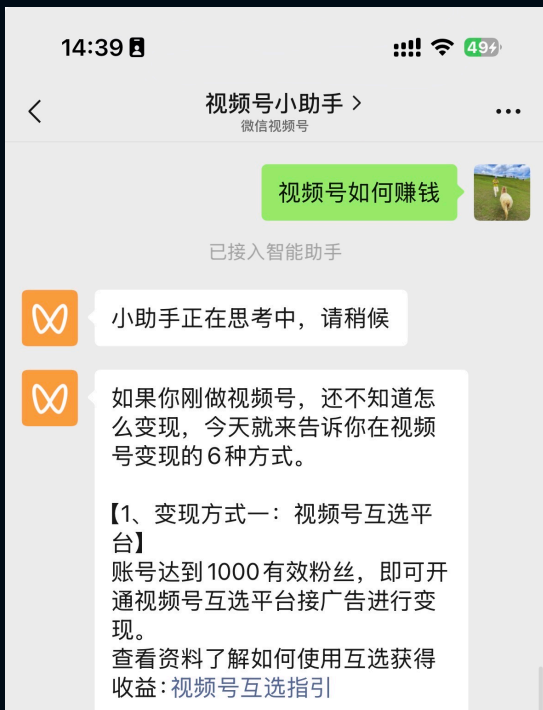
背景

Agentic 时代的到来

- Agent 广泛进入了我们我们日常的生活工作中
- 软件交互范式变革:
 - 人操作工具 -> Agent 自主决策与执行
- Agent 核心能力:
 - 感知 -> 记忆 -> 推理 -> 行动
- OLAP 基础设施转型:
 - 服务人 -> 服务 Agent



微信常见 Agent 业务场景



平台客服



领域助手



智能推荐

挑战

- *Agent* 可观测性
 - 传统可观测工具在 *Agent* 场景失效
- 记忆存储/检索
 - *Agent* 工作需要依赖于高效地记忆存储检索和召回
- *Agent* 访问保护
 - OLAP 访问主体从“人”变成“*Agent*”，如何进行访问保护

02

Agent 可观测性探索

Agent 可观测 VS 传统可观测

观测对象变化：系统状态 -> 决策过程

传统可观测：

为确定性系统设计：路径固定，指标固定，成功标准固定

固定拓扑

调用路径部署时确定

系统指标

QPS/延迟/错误率

技术成功

HTTP 200即视为正常

失效

Agent不是一条固定链路，而是一段动态决策

Agent场景四个结构性盲区

01 动态路径

无法提前埋运行
时生成点

02 成本不透明

Token随上下文滚雪球，难归因

03 静默偏差

技术成功但语义错误

04 多轮交互

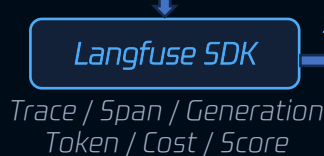
Trace与上下文数据爆炸

Agent 可观测：不止是“监控”，而是“理解”：Agent 走了哪条路？做的对不对？钱花在哪里？数据是否撑得住？

Langfuse + OLAP

• Langfuse + OLAP: 构建大模型 Agent 可观测系统

数据产生侧



上报

LANGFUSE

开源 LLM 可观测平台

功能层: 把 Agent 的运行过程变成可解释、可评估、可优化的数据

Trace 追踪

User → Session → Trace
Generation / Span / Tool
Call

Evaluation 评估

LLM-as-a-Judge
正确性 · 完整性 · 幻觉

Prompt / Dataset

Prompt 管理
数据集与实验样本

Dashboard / Cost

延迟 · Token · 成本
质量趋势与问题定位

Trace 数据落盘

ClickHouse

OLAP 数据底座 · 高吞吐写入 · 多维分析

可观测结果

走了哪条路?

完整还原动态执行链路

做得对不对?

LLM 评估 + 人工校准

钱花在哪里?

Token / Cost 逐层归因

分析结果

Langfuse Agent观测平台

DataFun.

AGENTIC AI
超级智能体系架构峰会

- 面向对轮对话设计，实现过程可视化、可评估、可回放，把黑盒变透明

The screenshot displays the Langfuse Agent observation platform interface, showing a detailed view of a trace for the agent 'kalm-harness'.

Left Panel: Trace List

- Header: 轨迹追踪 (Trace Tracking)
- Filters: 隐藏筛选 (Hide Filter), 视图 (View), 搜索... (Search...), IDs / Names
- Table Columns: 时间 (Time), 名称 (Name)
- Table Content:

时间	名称
2026-05-29 15:13:48	kalm-harness
2026-05-29 15:12:33	kalm-harness
2026-05-29 15:11:46	kalm-harness
2026-05-29 15:11:24	kalm-harness
2026-05-29 15:10:49	kalm-harness
2026-05-29 15:10:13	kalm-harness
2026-05-29 15:06:30	kalm-harness
2026-05-29 15:04:07	kalm-harness
2026-05-29 15:03:58	kalm-harness
2026-05-29 14:53:24	kalm-harness
2026-05-29 14:52:34	kalm-harness
2026-05-29 14:38:25	kalm-harness
2026-05-29 14:38:12	kalm-harness
2026-05-29 14:37:46	kalm-harness
2026-05-29 14:37:31	kalm-harness
2026-05-29 14:37:00	kalm-harness
2026-05-29 14:36:47	kalm-harness
2026-05-29 14:36:39	kalm-harness
2026-05-29 14:36:01	kalm-harness
2026-05-29 14:36:01	kalm-harness
2026-05-29 14:35:05	kalm-harness
2026-05-29 14:34:48	kalm-harness
2026-05-29 14:34:48	kalm-harness

Trace Details (Trace ID: 505d85796d0244878bfe2b8dbcd2352)

- Search: [Search Bar]
- Timeline: [Timeline View]
- Trace Structure:

```
graph TD
    start([start]) --> agent_resolve[agent_resolve]
    agent_resolve --> sandbox_setup[sandbox_setup]
    sandbox_setup --> session[session]
    session --> iteration_3_3[iteration (3/3)]
    iteration_3_3 --> start
```

Trace Details (kalm-harness ID)

- Latency: 9.74s
- Session: engine-7364-4ff27b0c
- User ID: [User ID]
- Env: dev
- Release: v0.0.3
- 7,736 prompt → 189 completion (Σ 7,925)
- Preview | Log View | Scores
- Formatted: JSON
- Input: 创建一个 python 脚本，打印出 hello
- Output:
Python 脚本创建成功!
我为您创建了 hello.py 文件，内容如下:

```
python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
print("hello")
```


脚本运行测试通过，成功输出了 hello。
您可以通过以下方式运行该脚本：
• python3 hello.py
• 或直接执行: ./hello.py (需先添加执行权限: chmod +x hello.py)
- Corrected Output (Beta): [Click to add corrected output]
- Metadata:

Path	Value
kalm.span_id	"c7037cf05e474d23b5d4670608cf42d8"
kalm.session_id	"engine-7364-4ff27b0c"
kalm.user_id	"[User ID]"
kalm.kind	"session"
kalm.latency_ms	6253
resourceAttributes	6 items
scope	3 items

为什么选择ClickHouse

Agent可观测数据：海量写入 X 半结构化 X 多维分析

挑战1: 大规模LLM 观测场景下, Trace 的输入输出包含海量自然语言大文本或半结构化 JSON 数据, 且通常需要长期留存

解决: 极致存储压缩: 列存 + 稀疏索引 + 多级压缩/稀疏编码 + 冷热分层

- 数据存储成本降低至传统方案 1/10

挑战2: 观测数据包含大量随业务变化的扩展属性, 如 metadata、tool calls 等, 字段不固定, 结构差异性大

解决: 原生半结构化类型 JSON/MAP 等支持:

- 兼顾扩展属性的灵活接入和高性能多维查询

挑战3: 可观测平台承载的查询具有强烈 OLAP 属性, 如时间窗口剪裁、多维度联合过滤、高吞吐指标聚合等, 低延迟要求

解决: 极致查询性能, 极致 IO 剪裁 + 先进执行引擎设计 + 稀疏索引/主键索引剪裁

社区版 Langfuse 弊端

- 仅支持 ClickHouse 单分片存储
 - 存储、写入吞吐、大数据量下的查询性能都受限于单机性能
 - 微信业务场景, Agent 调用频率高、产生数据量大, 单机瓶颈突显

Support Multi Shard Clickhouse DB for Langfuse #12158

Closed

RahulMR42 started this conversation in Ideas



RahulMR42 on Feb 20

Describe the feature or potential improvement

What we think / aware that - Langfuse itself does not usually provide a native "shard routing" layer where it splits tenant/workload across multiple independent org/region/environment Clickhouse DBs

Support multi-shard clickhouse clusters #5021

Steffen911 started this conversation in Ideas

Additional information



Steffen911 on Jan 14, 2025

Maintainer

Describe the feature or potential improvement

Add support for multiple shard clusters in ClickHouse. As of today, those are not supported and lead to inconsistent query results as discussed in <https://github.com/orgs/langfuse/discussions/4950%5C>.

Additional information

No response

34

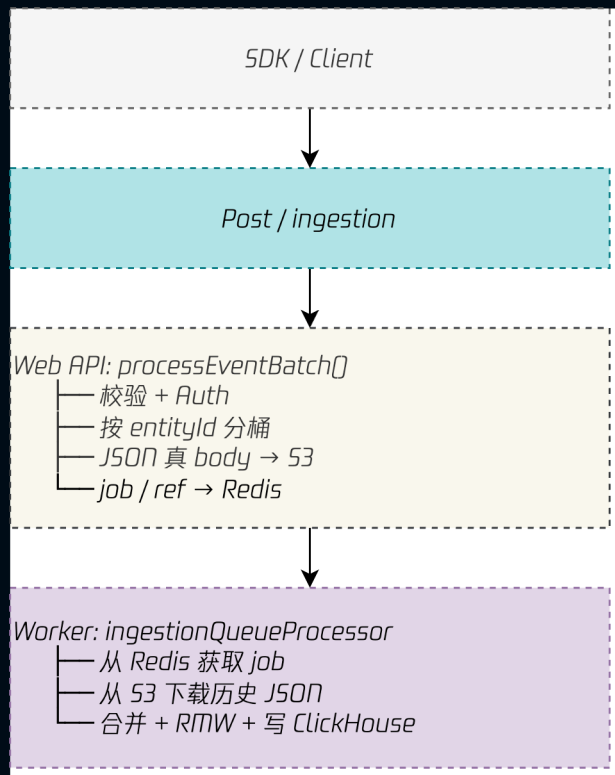


8

社区关于 ClickHouse 多 Shard 支持的讨论, 但迟迟没有进展

社区版 Langfuse 弊端

- 接入性能差
- Langfuse 原有接入链路基于 S3 历史数据回放 + RMW 来进行合并更新
 - 读放大, `listFiles`+全量下载+merge
 - 写入产生大量 ClickHouse 点查
- Redis 队列积压:
 - 高并发写入时, `ingestion queue`堆积
- S3 上传延迟
 - 每次 OTel 请求都先上传 JSON 到 S3

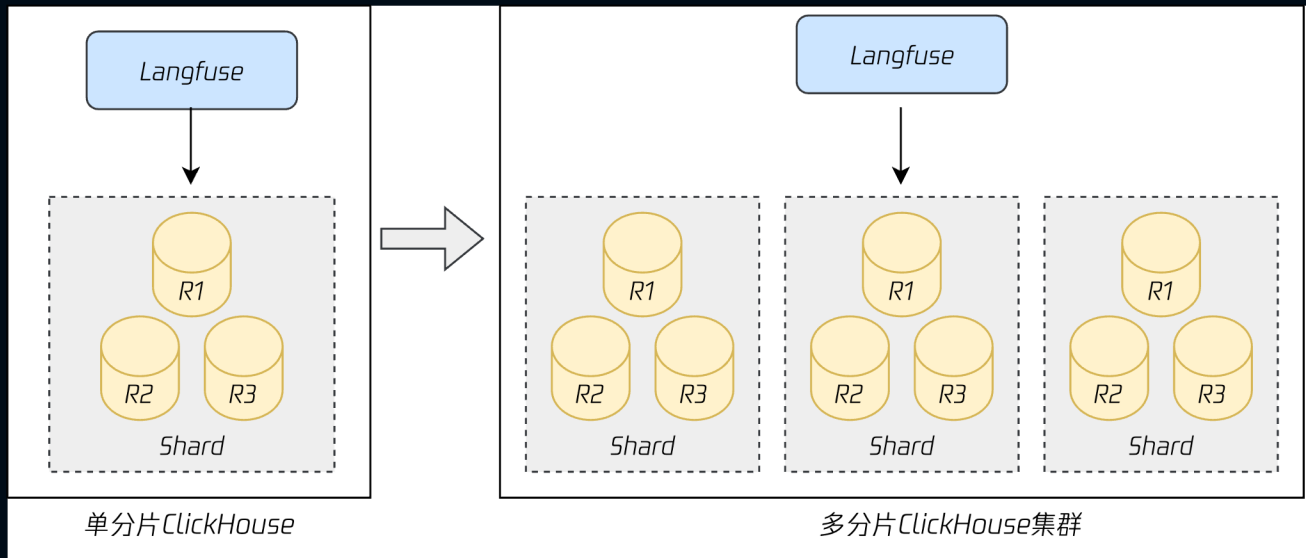


社区版 *Langfuse* 弊端

- 大数据量下查询性能退化，多模态点查性能差
- 表结构设计：
 - *Traces* 表主键粒度天级别，*id* 查询需要扫描一整天
 - 对 *input/output* 的 *ILIKE* 匹配需要全表扫描
- 双实体模型
 - *traces* + *observations* + *scores* 多表 JOIN
- 推理等多模态日志数据，视频号截帧数据存储为数仓中，列存点查性能差

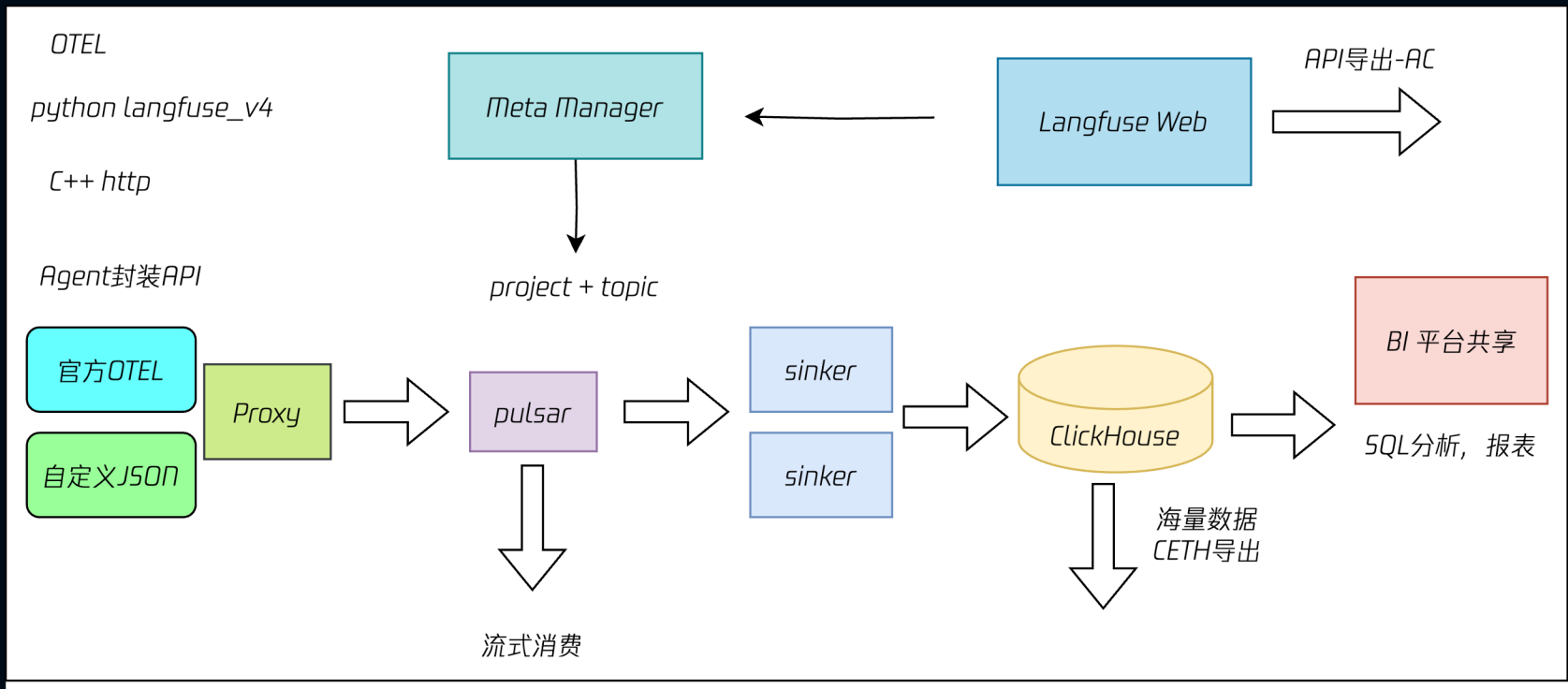
从单机到高可用/可扩展存储分析

- 分布式架构改造:
 - 消除单节点存储和写入物理瓶颈，写入和查询分析在物理上得到分散与解藕
 - 配合多副本的高可用和低成本冷热存储下沉，为大规模 Agent 调用可观测性构建了稳固、弹性且可线性扩展的高性能存储底座



高吞吐接入链路

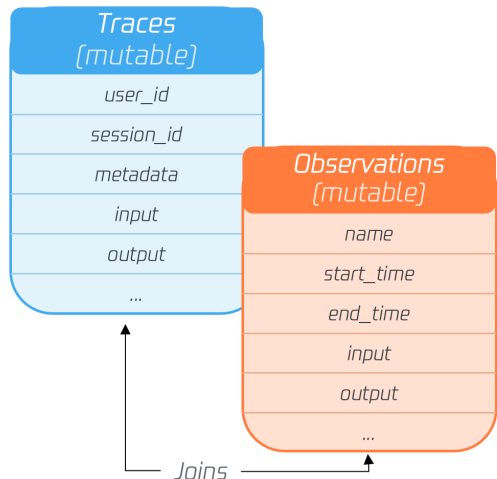
- 面向大规模 Agent 接入的高吞吐导入链路, V4 兼容



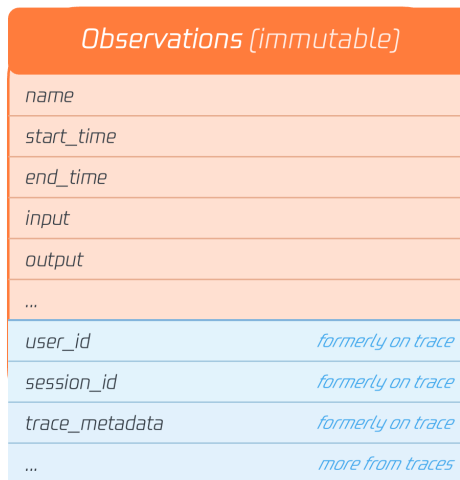
查询性能提升

- 迈向V4
 - Trace-centric 到 observations-centric
 - Immutable 模型

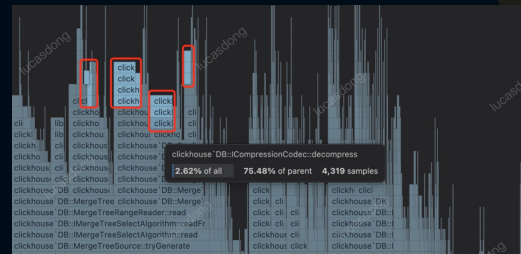
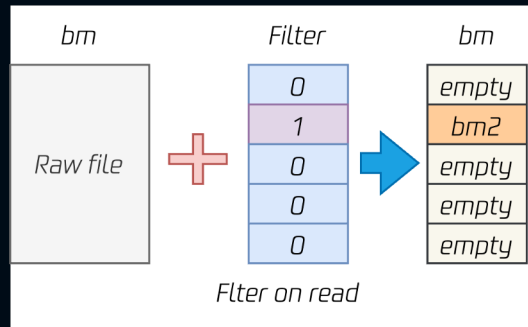
v3 Mutable traces & observations tables that have to be joined



v4 A single immutable observations table with new fields formerly stored on traces



- 内核增加行级索引和后过滤能力，提升点查性能



SDK 规范化上报

- *Trace* 数据除用于观测外，还用于
 - SFT 训练
 - 模型蒸馏训练
- 问题：
 - *Langfuse* SDK 版本多，接口种类多
 - 不同用户上报格式不统一
 - 不规范的上报导致后续难以利用
- 解决：
 - 规范化上报标准和上报 SDK
 - 上报文档[Skill, 供AI写出符合规范上报代码

```
json
{
  "messages": [
    {
      "role": "user", "content": "台北現在天氣如何？",
      {
        "role": "assistant",
        "tool_calls": [
          {
            "id": "call_123",
            "type": "function",
            "function": {"name": "get_current_weather"}
          }
        ]
      },
    {
      "role": "tool", "tool_call_id": "call_123", "co
    {
      "role": "assistant", "content": "台北現在的氣溫是
    ]
  }
}
```

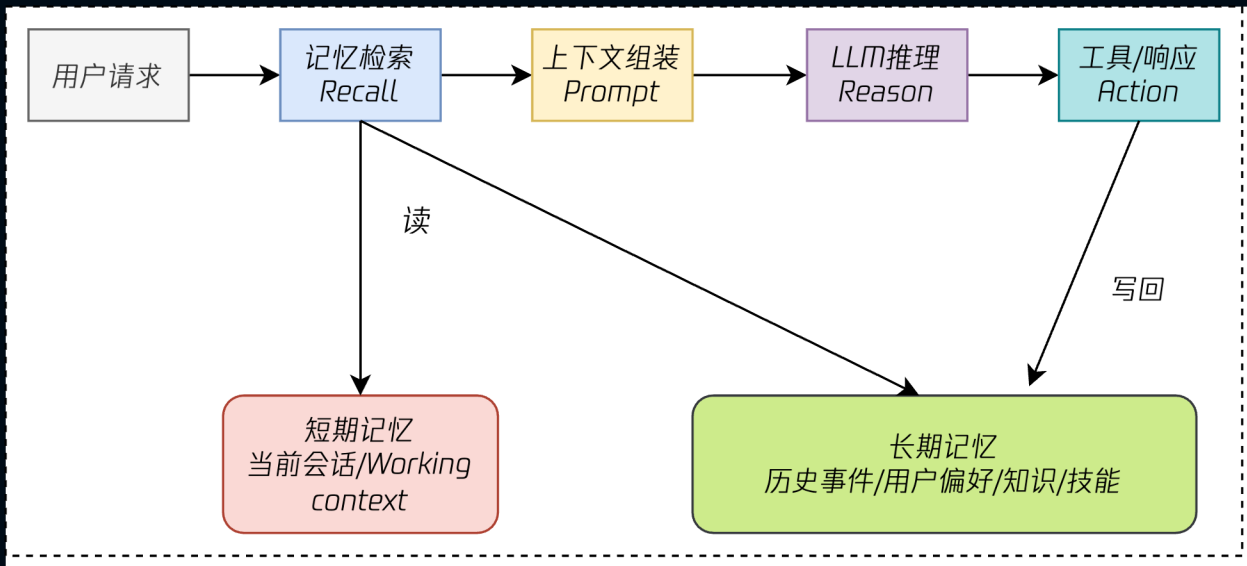
OpenAI 数据格式

03

Agent 记忆底座探索

为什么Agent 需要记忆

- 把历史信息变成当前决策可用的上下文

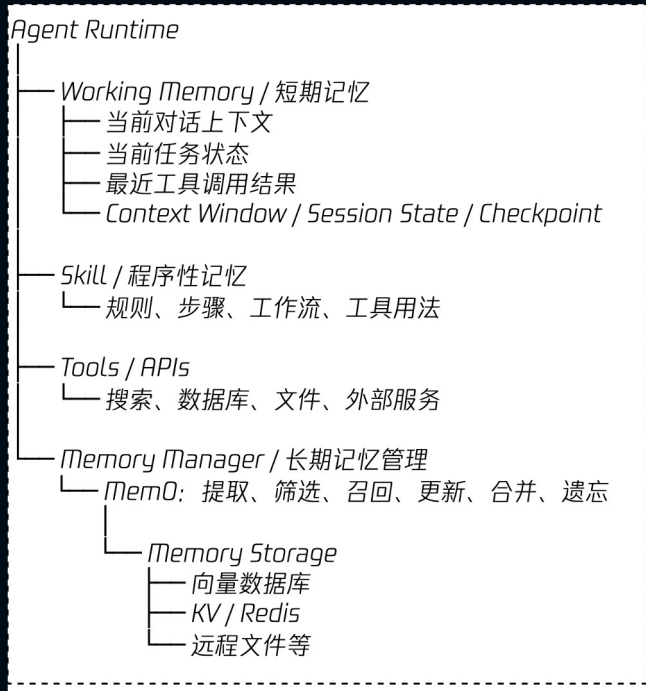


- ✓ 连续性: 跨轮次保持上下文
- ✓ 个性化: 记住用户偏好
- ✓ 高效率: 历史结果复用
- ✓ 可成长: 沉淀反馈与经验, Agent越用越好

没有记忆, 每次请求都是第一次; 有了记忆, Agent 才能持续服务

生产级Agent常见记忆组织形式

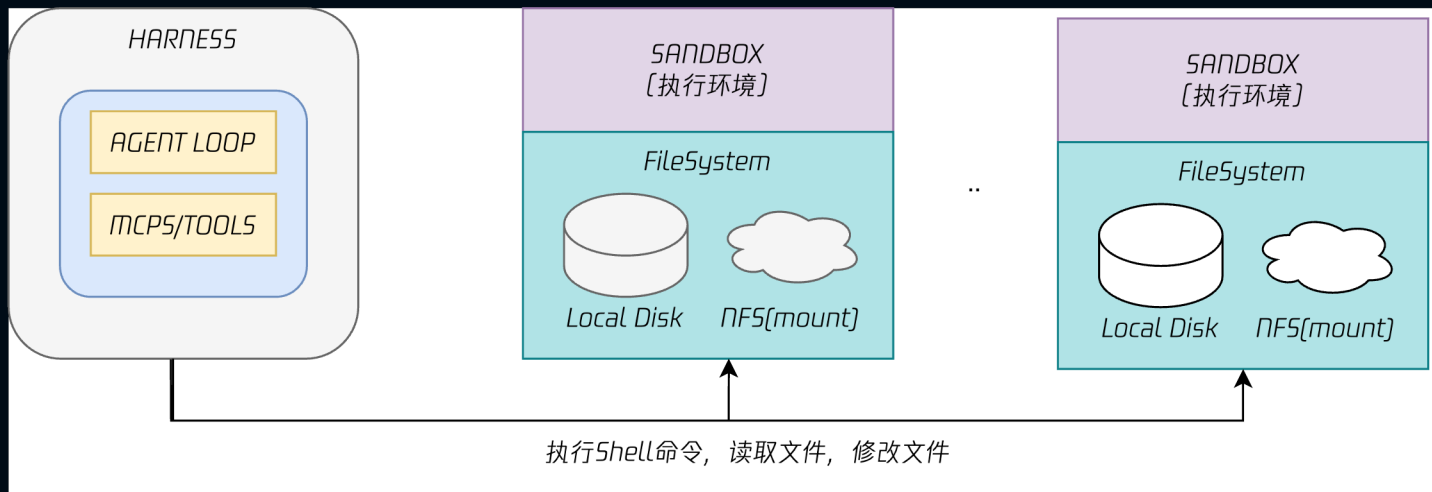
- 短期记忆
 - 当前任务正在发生什么?
 - 当前对话, 任务状态, 中间结果
- Skill
 - 这类任务应该怎么做?
 - 步骤, 规则, 工作流
- MemO
 - 哪些信息值得记住和召回
 - 用户事实、偏好、历史经历等
- Memory Storage
 - 记忆保存在哪里



算法同学关注记忆如何提取, 删选, 更新等;
我们关注如何提供更好的记忆存储服务

为什么记忆需要远程存储

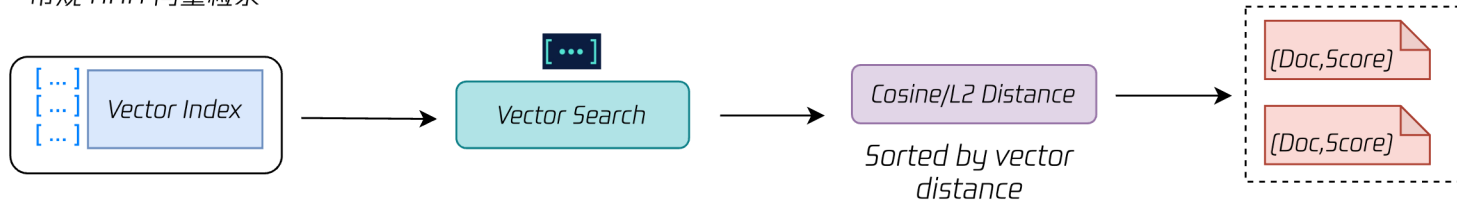
- 大规模运行的 Agent 实例通常使用沙箱作为执行环境
 - Agent 实例可以销毁、重启、扩容、迁移，但记忆不能跟着实例一起消失
 - 多Agent实例经常需要读取同一批文档、规则、代码或 RAG 语料
- 因此，需要远程存储保证Agent记忆的持久化



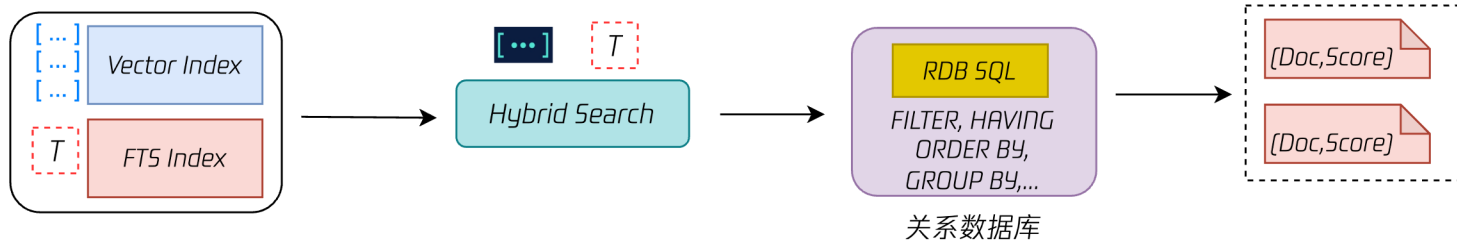
从OLAP数据库到Agent记忆底座

- OLAP 向量检索：不止于OLAP，BM 25 + ANN 混合检索 + 聚合分析 + 综合排序
- 一站式 SQL 分析能力：语义检索 + 结构化检索，单条 SQL 完成复杂过滤、检索、统计、排序

常规 ANN 向量检索



OLAP 向量检索：BM25 + ANN 混合检索 + 聚合分析 + 综合排序



从OLAP数据库到Agent记忆底座

- 针对亿级数据的磁盘向量索引DiskANN支持
 - 通过PQ量化对vector进行压缩，压缩后的vector放在内存中，graph和全精度向量放在SSD上

数据量	DiskANN	HNSW
400万	几百MB以内	14.8GB
1亿	7.5GB	270GB
10亿	68GB	

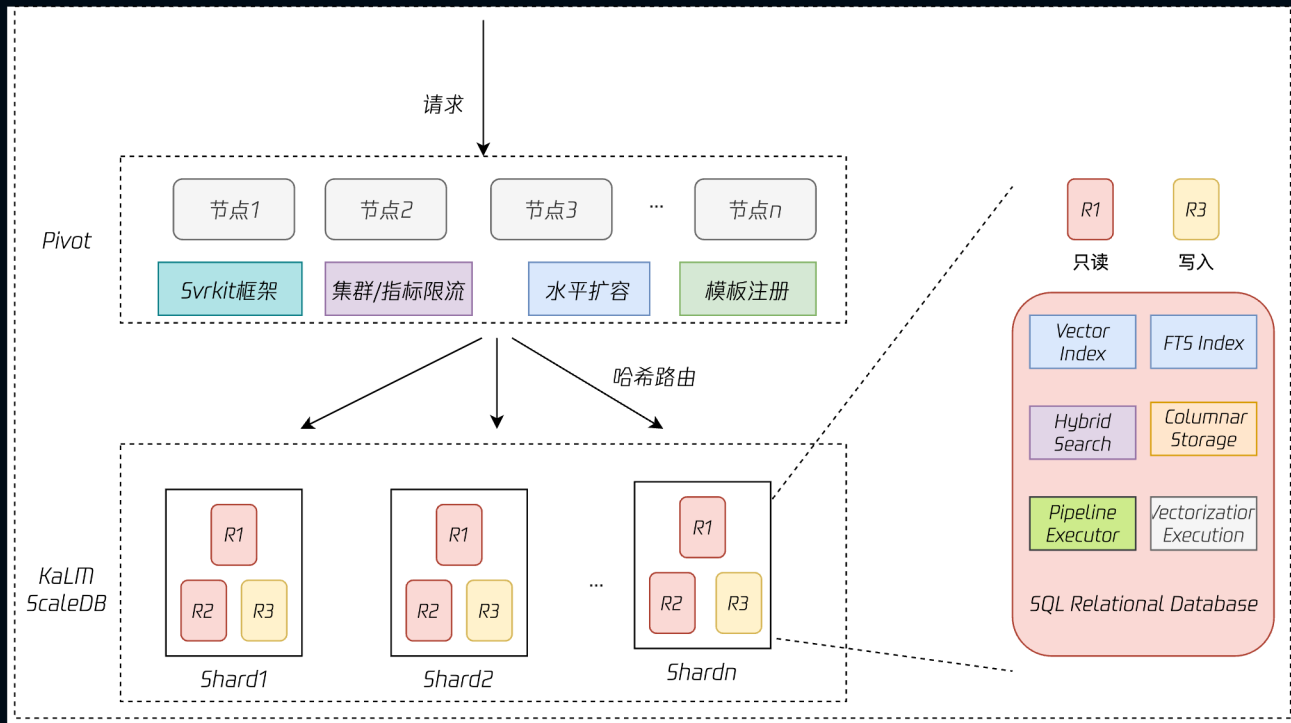
内存占用相比HNSW降低90%以上

数据量	DiskANN	HNSW
400万	377	506
1亿	371	518
10亿	60	

QPS降低40%

大规模 Agent 服务场景支持

- 多 Agent 在线服务场景对 QPS、延迟、稳定性提出更高要求

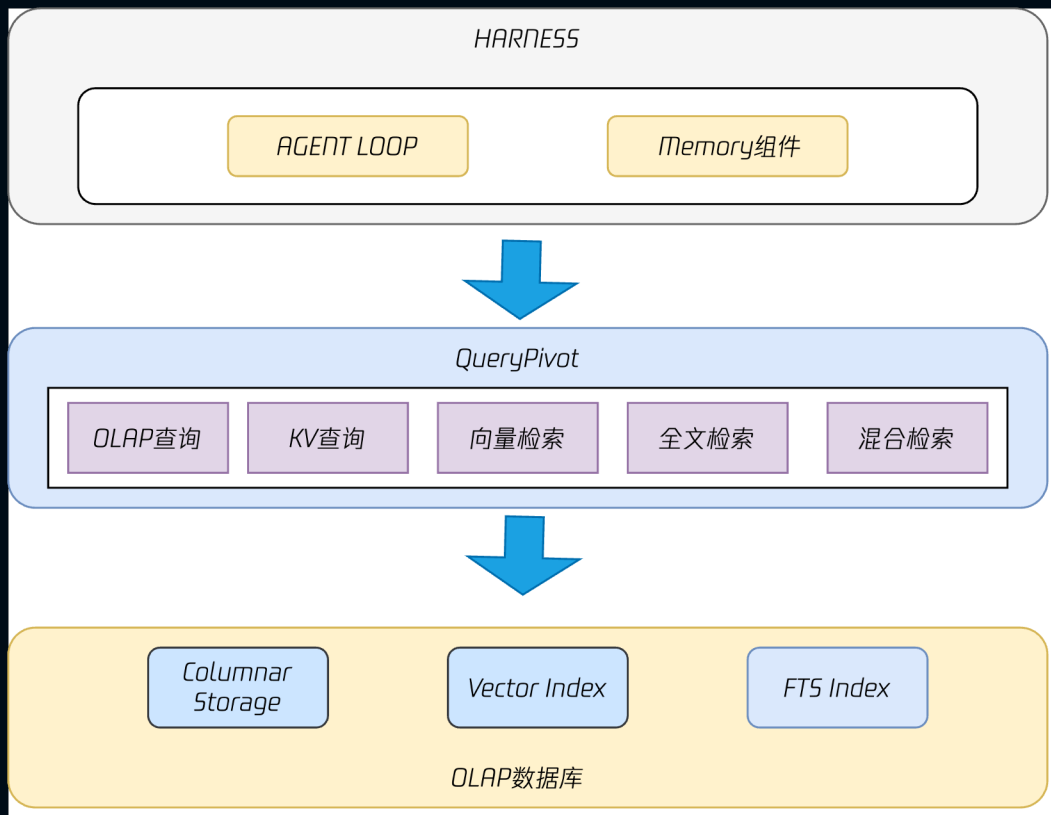


- ✓ 基于微信后台框架的高QPS路由架构
- ✓ 按Agent哈希路由
- ✓ 副本间读写分离

记忆服务

- *Memory API/SDK*
 - 提供通用 *Memory* 访问 *API/SDK*, 远程存储使用 *OLAP* 向量数据库
 - *Agent* 通过 *Memory API* 来访问远程存储
- 面向文件系统的记忆服务
 - 文件系统是 *Agent* 沙箱最自然、最统一的上下文访问接口
 - *Agent* 无需学习一套新的 *Memory API*, 简单易用

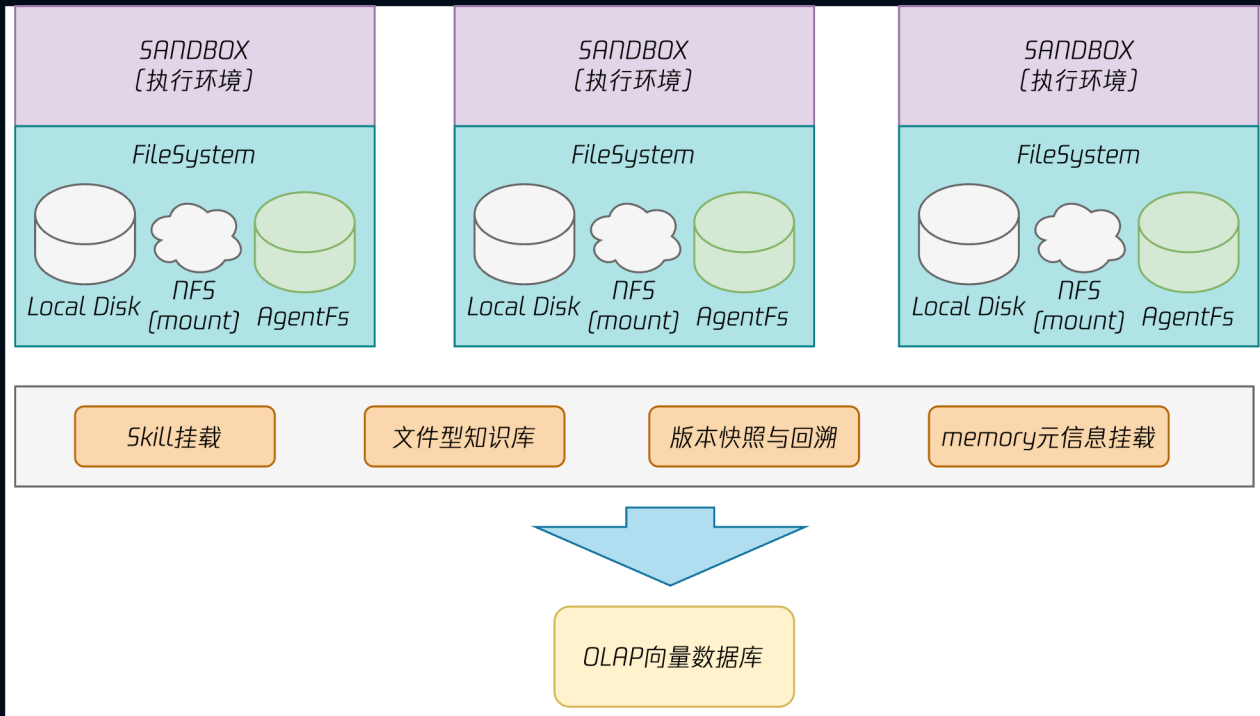
基于 OLAP 数据库的记忆服务



- 扩展整合OLAP向量数据库能力，为Agent Memory提供底座能力支持
- 提供通用记忆存储API

面向文件系统的Agent记忆存储

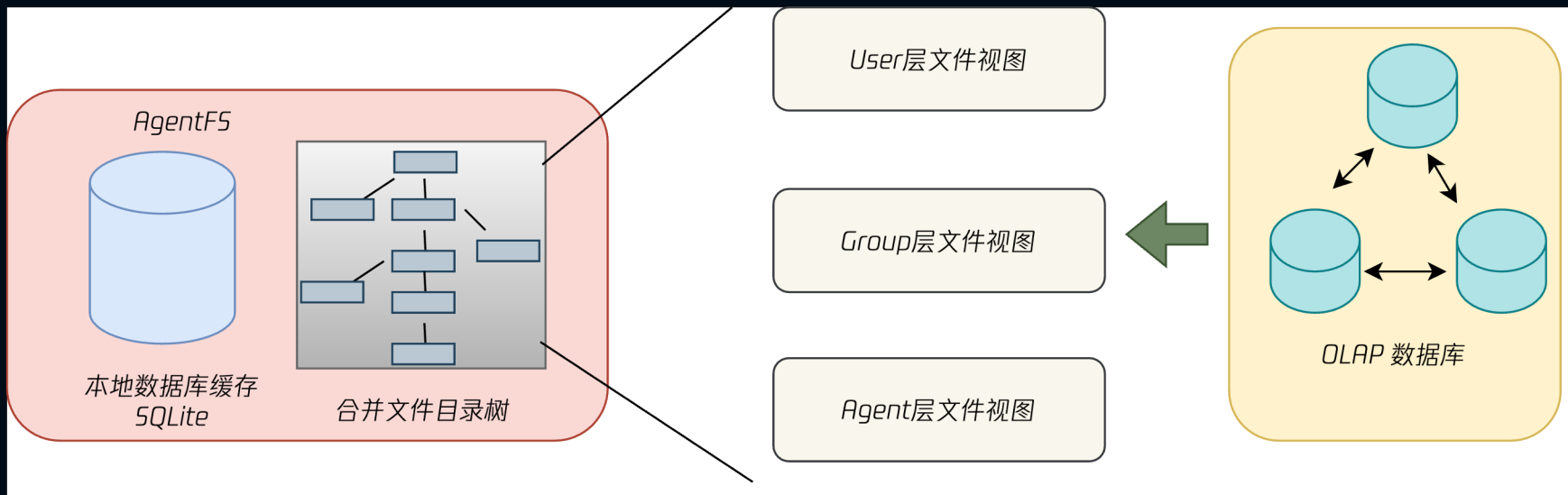
- 以 OLAP 向量数据库为远程存储，把文件形式组织的“上下文和记忆”，以文件形式挂载在沙箱执行环境中



- Agent 无需学习 SQL 语法，无需对接复杂数据库 API
- 以 LLM 最熟悉的命令来访问数据库

面向文件系统的Agent记忆存储

- 通过 OverlayFS 支持多层文件视图，通过指定不同的 [User, Group, Agent] 可以获取不同的文件目录树 [上层覆盖下层]



04

Agent 访问保护

从“人”访问到“Agent”访问

- 过去, “人”访问数据库
 - “有限次、有意图、可追责”的慢操作
- 现在, “Agent”访问数据库
 - “无限次、按规则、难归因”的自动化操作, “微小错误”可能“瞬间放大”
 - 权限滥用风险, Agent 访问常被过度授权
 - SQL注入与恶意载荷
 - 审计与追责困难, 数据泄露更加隐蔽

如何保护

- 把“Agent”当“人”看
 - 每个需要访问数据库的Agent，在上游进行“身份”注册，获取数据库访问权限
- 权限管控与审计
 - 数据库访问需要携带对应的“身份”信息和动态票据进行权限认证
 - 审计日志上报
- 限流
 - 默认限流，根据身份信息进行限流
 - 异常访问快速封禁

THANKS

演讲嘉宾：冯吕
